

# AIDS final report

Liran Napadenski

August 2025

## Web Application Attacks

In this work, I propose two new models and one based on a paper - for detecting web application attacks, specifically targeting SQL injection, Cross-Site Scripting (XSS), and other common attack types such as Command Injection, malicious form submissions, API misuse with crafted JSON payloads, and tampering with URL parameters.

### Characteristics

These types of attacks rely on exploiting the existing server infrastructure to execute malicious code or manipulate backend behavior. To do this, the attacker must inject crafted input into the server, typically in one of two ways:

- Through the URL, using query parameters (common in GET requests).
- Through the HTTP request body (common in POST, PUT, or PATCH requests).

### Other Web Attacks

In addition to URL or body-based injections, attackers exploit web applications through several other techniques:

- **Session Hijacking:** Stealing session identifiers from cookies or headers to impersonate a user.
- **Cross-Site Request Forgery (CSRF):** Forcing a user's browser to send authenticated requests by exploiting session tokens in cookies.
- **HTTP Header Injection:** Manipulating headers to perform attacks such as response splitting or cache poisoning.

These types of attacks are beyond the predictive capabilities of our models, as none of the models use headers or cookies, which are the primary vectors for such attacks.

## Characteristics in Log Records

Attacks on web applications can leave traces in different parts of HTTP requests, including the URL, the request body, headers, and cookies. Logs provide a partial view of these attacks, and their characteristics can help identify malicious activity even if the payload itself is not fully visible.

- **Malicious URL patterns:** URL-based attacks often include special characters or keywords such as `'`, `--`, `<script>`, or SQL keywords like `SELECT`, `UNION`, and `OR 1=1`. These can appear directly in the request line of access logs and may indicate SQL injection, XSS, or directory traversal attempts.
- **Suspicious request bodies:** Body-based attacks, common in POST, PUT, or PATCH requests, can include crafted payloads such as JSON or form parameters carrying SQL statements, script tags, or system commands. Examples include login forms with injected SQL or API endpoints with manipulated JSON/XML fields.
- **HTTP error codes:** Malicious inputs in the URL or body can trigger internal server errors (e.g., HTTP 500) or other abnormal responses. This can result from intentionally malformed inputs such as buffer overflows, division-by-zero triggers, or invalid parameter values. Such anomalies often appear in logs as signs of failed parsing, database errors, or server-side exceptions.
- **Unusual request patterns:** High-frequency requests from a single IP, repeated access to a specific endpoint with varying URL parameters or body content, and other automated patterns may indicate bot activity or an attack attempt.

By analyzing the URL, request body, and metadata such as headers, cookies, and request frequency, logs can provide hints of malicious activity even if the full payload is not visible. However, attacks that primarily rely on headers or cookies are outside the detection scope of models in this paper which are trained solely on URLs and request bodies this paper talks about.

## Dataset

All models in this study were trained and tested using the **CSIC 2010 Web Application Attacks dataset**, which is publicly available at <https://www.kaggle.com/datasets/ispangler/csic-2010-web-application-attacks>.

## Creation

The CSIC 2010 HTTP dataset simulates traffic for an e-commerce web application developed by the Information Security Institute of CSIC. Users can register and make purchases using a shopping cart. The dataset includes some Latin characters due to the web application’s Spanish origin.

To generate the dataset, real data for all parameters (names, surnames, addresses, etc.) were collected and stored in two separate databases: one for normal values and one for anomalous values. All publicly available web pages of the application were listed, and requests were generated for each page. Normal requests used values randomly drawn from the normal database, while anomalous requests used values from the anomalous database.

## Structure

The dataset is organized as a table with approximately 60,000 rows, each representing an HTTP request, and 17 columns, each describing different aspects of the request:

- **Method:** The HTTP method used (GET, POST, PUT, etc.).
- **URL:** The requested resource.
- **User-Agent:** Identifies the client’s browser and operating system.
- **Pragma:** HTTP/1.0 general-header affecting request-response handling.
- **Cache-Control:** Specifies browser and server caching policy.
- **Accept:** MIME types the client can understand.
- **Accept-Encoding:** Content encodings the client supports (e.g., gzip).
- **Accept-Charset:** Character sets acceptable in the server response (e.g., UTF-8).
- **Accept-Language:** Preferred language and locale.
- **Host:** Web domain being accessed.
- **Connection:** Controls whether the network connection stays open after the request.

- **Content-Length:** Size of the message body in bytes.
- **Content-Type:** Media type of the resource.
- **Cookie:** User-sent HTTP request headers.
- **Payload:** Includes headers, URL, body content, and request/response status information.
- **Label:** Indicates whether the request is normal or an attack.

## Features Used

All three models utilize the same key features from HTTP requests:

### 1. URL

- **Description:** The Uniform Resource Identifier (URI) in an HTTP request identifies resources on the web. It typically includes the path and query parameters that specify the requested resource and any additional data sent via the URL.
- **Relevance:** Attackers often manipulate URL paths and query parameters to deliver malicious payloads. Monitoring anomalies or suspicious patterns in the URI can reveal attacks such as SQL injection, directory traversal, and cross-site scripting (XSS).

### 2. Request Body

- **Description:** The request body carries client-sent data, primarily in POST or PUT requests. It often contains user input, form submissions, JSON payloads, or other data.
- **Relevance:** The request body is a primary vector for injection attacks, as it contains most user-supplied data. Malicious scripts, commands, or malformed data frequently appear here, making it critical for accurate detection.

**\*Extraction Process:** Both the URL and the request body are provided as separate fields in the CSIC 2010 dataset. They can be directly accessed for model training and testing

# 1. Character Transformer

\*Before the content is fed into the model, it passes through a one-hot vector tokenizer that converts each character into a one-hot representation

## structure

The model consists of two parts - character level encoder based on "Attention is all you need" [2], In more detail:

1. an embedding layer
2. positional encoding - add a value to each vector in the sequence in accordance to its order
3. a number of identical layers composed of :
  - self attention - to give more weight and focus on the more important parts of the sequence (chars)
  - Add + Norm: adding the output of the self-attention to the input, then applying a normalization layer. This ensures stable values and prevents gradient explosions that could introduce noise. The addition step also allows the model the option to 'ignore' the attention
  - MLP which in my case is :
    - Linear to higher dimension
    - ReLU
    - Drop
    - Linear back to the same dimension
  - another add + norm

and the second part is a MLP which consists of only one Linear layer

## parameters

- **number of layers** in the encoder -Increasing the number of layers in the encoder allows the model to build more complex representations and apply attention more effectively to important parts of the input

| Number of Layers | Accuracy      | Precision      | Recall        | F1 Score      |
|------------------|---------------|----------------|---------------|---------------|
| 2                | 99.65%        | 99.89%         | 99.58%        | 99.74%        |
| 3                | 99.79%        | <b>100.00%</b> | 99.69%        | 99.84%        |
| 4                | <b>99.93%</b> | 99.98%         | <b>99.91%</b> | <b>99.94%</b> |

Table 1: Model performance with different numbers of layers

therefore `number_of_layers=4`

- **Dropout** Drop (set to 0) a vector in a sequence with probability equal to the dropout rate. This prevents the model from over-relying on specific inputs and helps improve generalization by encouraging redundancy and robustness in learned features. but you drop too much and your sequence loses all meaning

| Dropout | Accuracy      | Precision      | Recall        | F1 Score      |
|---------|---------------|----------------|---------------|---------------|
| 0.1     | <b>99.93%</b> | 99.98%         | <b>99.91%</b> | <b>99.94%</b> |
| 0.2     | 99.79%        | <b>100.00%</b> | 99.68%        | 99.84%        |

Table 2: Model performance with different dropout

The recall is the most important factor as one breach would do much more damaged then rejected normal request, therefore the optimal choice is `dropout=0.1`

- `embedding_dim=64` : This embedding dimension offers a good balance between capturing character semantics and preventing overfitting on limited data.
- `max_length = 200`: Limits input length to maintain computational efficiency while covering typical request sizes.
- `num_of_heads = 8`: I use multi-head attention with 8 heads. This increases the model's expressiveness by allowing it to focus on different parts of the input sequence simultaneously.

## training and testing

First, I used `train_test_split` of `sklearn` to split the dataset to tow parts one for training(0.8) and the other for testing(0.2)

For training I used ADAM optimizer with learning rate of `lr=1e-4` over 400 batches(epoches) with 32 random samples in each batch, and CrossEntropy as loss function. the training took approximately 20s per epoch

For evaluation I run on all the testing dataset and predicated the classification and calculated the metrics using the labels and predictions

## results

| Accuracy | Precision | Recall | F1 Score |
|----------|-----------|--------|----------|
| 99.93%   | 99.98%    | 99.91% | 99.94%   |

Table 3: best Model performance

## mistake analysis

1. `http://localhost:8080/tienda1/publico/anadir.jsp HTTP/1.1`  
`id=1&nombre=Jam%F3n+Ib%E9rico&precio=39&cantidad=3&B1=A%F1adir+al+carrito`

This request is malicious but was incorrectly predicted as benign. A quick inspection of the URL and request body reveals no clear indicators of an attack. Therefore, the most likely possibilities are that the malicious component resides in the headers or cookies, or that an illegal value was passed to a parameter. Since our model relies solely on the URL and request body, it has no way of detecting such attacks.

2. `http://localhost:8080/tienda1/publico/anadir.jsp`  
`HTTP/1.1id=3\&nombre=Vino+Rioja`  
`\&precio=39\&cantidad=66\&B1=A\%F1adir+al+carrito`

This request is benign but was classified as malicious. It is structurally similar to the previous request, highlighting how attacks that rely on headers or cookies can cause confusion for models that only analyze the URL and request body. During training, when the model encounters requests where the URL and body appear normal but the request is labeled as malicious, it may incorrectly learn to associate similar benign patterns with attacks.

## 2. Bidirectional LSTM

\*Before the content is fed into the model, it passes through a one-hot vector tokenizer that converts each character into a one-hot representation

### structure

1. an embedding layer
2. bidirectional LSTM layers based on[1]
3. MLP on the last hidden state which includes 2 Linear layers and ReLU in between

### parameters

- **Number of LSTM layers:** Increasing the number of LSTM layers allows the model to learn more complex and hierarchical temporal patterns, potentially improving performance on sequence tasks. However, deeper networks are harder to train, may overfit if not regularized, and can suffer from vanishing gradients or increased computational cost.

| Number of Layers | Accuracy | Precision | Recall | F1 Score |
|------------------|----------|-----------|--------|----------|
| 2                | 97.50%   | 100.00%   | 95.98% | 97.95%   |
| 3                | 97.50%   | 100.00%   | 95.98% | 97.95%   |

Table 4: Model performance with different numbers of layers

therefore `num_of_layers=2`

- **Dropout**

| dropout | Accuracy | Precision | Recall | F1 Score |
|---------|----------|-----------|--------|----------|
| 0       | 97.50%   | 99.59%    | 96.39% | 97.96%   |
| 0.1     | 97.50%   | 100.00%   | 95.98% | 97.95%   |
| 0.2     | 97.16%   | 99.05%    | 96.31% | 97.66%   |

Table 5: Model performance with different dropout

A dropout rate of 0 performed best, as it achieved the highest recall and F1 score

- **embedding\_dim=64 :** This embedding dimension offers a good balance between capturing character semantics and preventing overfitting on limited data.
- **max\_length = 100:** Limits input length to maintain computational efficiency while covering typical request sizes.



## training and testing

First, I used `train_test_split` of `sklearn` to split the dataset to two parts one for training(0.8) and the other for testing(0.2)

For training I used ADAM optimizer with learning rate of `lr=1e-4` over 400 batches(epoques) with 16 random samples in each batch, and CrossEntropy as loss function. the training took approximately 60s per epoch

For evaluation I run on all the testing dataset and predicated the classification and calculated the metrics using the labels and predictions

## results

| Accuracy | Precision | Recall | F1 Score |
|----------|-----------|--------|----------|
| 97.50%   | 99.59%    | 96.39% | 97.96%   |

Table 6: best Model performance

## mistake analysis

1. `http://localhost:8080/tienda1/publico/anadir.jsp HTTP/1.1`  
`id=1&nombre=Vino+Rioja&precio=85&cantidad=`

This request is malicious but was incorrectly predicted as benign. The absence of a value for the `cantidad` parameter is due to the model's maximum input length of 100 characters, which truncated the request. Even with this truncation, no suspicious patterns are visible in the URL or body. The malicious component likely resides in the headers or cookies, making it undetectable by models that only analyze the request body and URL.

2. `http://localhost:8080/tienda1/publico/anadir.jsp`  
`id=3&nombre=Jam%F3n+Ib%E9rico&precio=39&cantidad=82`

This request is benign but was incorrectly classified as malicious. Its structure is typical of a normal shopping-cart addition, and no attack signatures appear in the URL or request body. The misclassification highlights how headers and cookies can introduce ambiguity, causing the model to confuse structurally similar benign and malicious requests.

## Comparing the 3 models

### results

| Model               | Accuracy | Precision | Recall | F1 Score |
|---------------------|----------|-----------|--------|----------|
| paper               | 99.84%   | 100.00%   | 98.29% | 99.10%   |
| paper based         | 93.23%   | 100.00%   | 88.39% | 93.84%   |
| LSTM                | 97.50%   | 99.59%    | 96.39% | 97.96%   |
| Transformer encoder | 99.93%   | 99.98%    | 99.91% | 99.94%   |

Table 7: Models comparison

The best-performing model was the encoder-based architecture, as transformers handle text better than RNNs (LSTM) and CNNs (the paper-based model). Unlike RNNs, transformers can attend to all parts of the sequence regardless of length, and unlike CNNs, they effectively capture long-range dependencies. Additionally, the model from the original paper focused more on identifying the specific malicious segment within the request, which may have hindered its overall classification performance.

### Shared Mistakes

All models incorrectly classified the following request as benign:

```
http://localhost:8080/tienda1/publico/anadir.jsp HTTP/1.1
id=1&nombre=Jam%F3n+Ib%E9rico&precio=39&cantidad=3&B1=A%F1adir+al+carrito
```

From the URL and request body alone, there are no clear indicators of an attack. This suggests that the malicious component likely resides in the headers, cookies, or in parameter values not visible here. Since all models rely solely on the URL and request body, they inherently lack the ability to detect such attacks. Additional examples of misclassified malicious requests:

```
http://localhost:8080/tienda1/publico/pagar.jsp HTTP/1.1
modo=insertar&precio=4014&B1=Pasas+por+caja
```

To make matters more confusing, similar benign requests exist, such as:

```
http://localhost:8080/tienda1/publico/pagar.jsp?modo=insertar&precio=5628&B1=Pasas+por+caja
```

These nearly identical requests with different labels introduce ambiguity, which makes it difficult for the models to distinguish between benign and malicious behavior. As a result, the model may incorrectly classify some benign requests as malicious and vice versa.

## different mistakes

1. `http://localhost:8080/tienda1/publico/registro.jsp  
modo=registro&login=sarge&password=apad-r8nad5ra&`

This request is malicious. The Transformer-based model correctly classified it as such, while the LSTM model failed to do so. A likely reason for the LSTM's mistake is that it struggles to capture long-range dependencies and subtle contextual patterns within the input. The suspicious parameters, such as the crafted `password` field, may appear benign in isolation, but their combination indicates an attack. Transformer models, with their attention mechanisms, are better equipped to identify these contextual relationships, leading to more accurate detection. also the LSTM model handles shorter input so if the the malicious part is outside of its range it may not detect him.

## Conclusion

The examined cases highlight two primary sources of misclassification across the models. First, several attacks relied on malicious components hidden in headers or cookies rather than the URL or request body. Since all models were trained exclusively on these two fields, they inherently lack the ability to detect such attacks. This structural limitation explains why certain requests with no obvious malicious indicators were consistently misclassified.

Second, differences between model architectures further influenced classification performance. The LSTM and paper-based models struggled with inputs requiring long-range contextual awareness or subtle distinctions, leading it to incorrectly classify some malicious requests as benign. In contrast, the Transformer-based model, with its attention mechanism, was better at capturing these dependencies and therefore achieved higher accuracy in such cases.

Finally, the presence of structurally similar benign and malicious requests added further complexity. Models faced confusion when nearly identical inputs carried different labels, reinforcing the need for broader feature coverage (e.g., incorporating headers, cookies, and metadata) in order to reduce ambiguity and improve robustness.

## References

- [1] Hasim Sak, Andrew Senior, and Françoise Beaufays. Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition. In *INTERSPEECH*, 2014.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.